Separation of Concerns (SoC) – Rövid összefoglaló

Alapelvek:

- **Kód szétválasztás**: Kódot a felelősségi körök alapján különítsük el.
- **Laza csatolás**: Külön felelősségi körök minimálisan függjenek egymástól.
- **Nagy kohézió**: Egy felelősségi körön belüli elemek szorosan kapcsolódjanak.

Előnyök:

1. Könnyebb érthetőség és átláthatóság.
2. Külön fejleszthető és karbantartható modulok.
3. Bővíthetőség és újrafelhasználhatóság.
4. Független unit tesztelhetőség.

Rétegek és architektúrák

1. Kétrétegű (UI - Data Access):

- Egyszerűbb alkalmazásokhoz.
- Nem ajánlott jelentős üzleti logika esetén.

2. Háromrétegű (UI - Business Logic - Data Access):

- **UI Layer**: Megjelenítés és interakciók.
- **Business Logic Layer**: Validációk, folyamatok, számítások.
- **Data Access Layer**: CRUD műveletek.

Architektúrák

MVVM (Models-Views-ViewModels):

- **View**: UI elemek és interakciók.
- **ViewModel**: Megjelenítési logika, adatkötés, események kezelése.
- **Model**: Adatosztályok, logika, adatok kezelése.
- **Előnyök**: SoC előnyei, UI logika leválasztva.
- **Hátrányok**: Extra komplexitás.

MVC (Models-Views-Controllers):

- **Model**: Adatkezelés és logika.

- **View**: Adatok megjelenítése.
- **Controller**: Felhasználói bemenet kezelése, Model és View összekapcsolása.
- **Előnyök**: SoC előnyei, keretrendszer támogatás.
- **Hátrányok**: Extra komplexitás, navigáció kezelése.

DV (Document-View):

- **Document**: Adatok tárolása és kezelése.
- **View**: Adatok megjelenítése, felhasználói interakciók.
- **Előnyök**: Több dokumentum/nézet támogatás.
- **Hátrányok**: Csak indokolt esetben használjuk, komplex.

Kódszervezés:

- **Java**: Package.
- **.NET**: Namespace, mappák.
- **Többrétegű nézetek**:
  - Használati eset (funkciók).
  - Logikai nézet (tervezés).
  - Implementációs nézet (komponensek).
  - Processz nézet (teljesítmény, skálázhatóság).
  - Telepítési nézet (telepíthető elemek).

Szoftverfejlesztés Fázisai

Fázisok:

1. **Specifikáció készítés, követelmények definiálása:**
   - Követelmények összegyűjtése és rögzítése.
   - Végtermék: követelmény specifikáció, Use Case modellek.
2. **Tervezés:**
   - Rendszerterv készítése (architektúra, egységek kapcsolatai).
   - Végtermék: rendszerterv, UML modellek.
3. **Implementálás:**
   - Kód fejlesztése.
   - Végtermék: forráskód, forráskód-dokumentáció.
4. **Hibakeresés:**
   - Hibatípusok: implementációs hiba, kommunikációs hiba stb.
   - Végtermék: javított forráskód.
5. **Tesztelés:**
   - Minőség-ellenőrzés, hibák javítása.
   - Végtermék: tesztelt kód, tesztelési dokumentáció.
6. **Üzembe helyezés:**
   - Telepítés, biztosított verziók kiadása.
   - Végtermék: működő alkalmazás.
7. **Üzemeltetés, karbantartás:**
   - Hibák javítása, frissítések.

- Végtermék: karbantartott alkalmazás, karbantartási dokumentáció.

Dokumentációk Típusai:

1. **Folyamatdokumentáció:**
   - Fejlesztés folyamatait rögzíti (pl. projektterv, jegyzetek).
2. **Termékdokumentáció:**
   - Szoftvertermék leírása (pl. követelmény specifikáció, rendszerterv).
3. **Felhasználói dokumentáció:**
   - Végfelhasználóknak szól, használati útmutató.
4. **Rendszerdokumentáció:**
   - Leghosszabb életciklus, tartalma:
     - Követelmény specifikáció.
     - Architektúra (UML csomag-, komponensdiagramok).
     - Tervezési részletek (statikus és dinamikus tervek).
     - Felhasznált algoritmusok, interfészleírás.
5. **Forráskód dokumentáció:**
   - Fejlesztőknek szól, kód működésének magyarázata.
   - Tartalom: keretrendszerek, API-k, tervezési minták.
6. **Tesztelési dokumentáció:**
   - Teszterv (stratégiák, prioritások).
   - Tesztesetek részletes leírása.
7. **Karbantartási dokumentáció:**
   - Üzemeltetési hibák, kisebb fejlesztések, változások.

Tesztelés jelentősége

1. **Tesztelés szükségessége**: Ellenőrzi a szoftver minőségét és működését.
2. **Dokumentáció részei**:
   - **Bevezetés**: Cél, módszertan, előnyök.
   - **Fogalomtár**: Technikai alapok.
   - **Architektúra és tervezés**: Magas és alacsony szintű tervekkel (pl. UML diagramok).
   - **Tesztelési dokumentáció**: Stratégiák, tesztek, ellenőrzőlista.
   - **Felhasználói és fejlesztői dokumentáció**: Használati útmutató, forráskód magyarázat.

Tesztelési megközelítések

1. **Automatikus vs manuális**: Gépi vagy emberi tesztelés, esetenként kombinálva.
2. **Blackbox vs whitebox**: Külső eredmények vagy belső működés elemzése.
3. **Funkcionális tesztek**: Minél több funkció lefedése minimális tesztekkel.

Teszttípusok

1. **Unit teszt**: Egyes metódusok helyessége.
2. **Modul teszt**: Teljesítmény, szinkronizáció vizsgálata.
3. **Integrációs teszt**: Komponensek kapcsolódása.

4. **Rendszerteszt**: Kész szoftver tesztelése.
5. **Átvételi teszt**: Felhasználók és rendszergazdák tesztelése.
6. **Regressziós teszt**: Korábbi hibák újbóli ellenőrzése.
7. **Biztonsági és terhelési tesztek**.

Szerződések és kockázatok

1. **Fix költségű szerződés**:
   - **Előnyök**: Stabil költség, előre rögzített funkciók.
   - **Hátrányok**: Rugalmatlanság, specifikáció hiányosságai.
   - **Kockázatcsökkentés**: Barter, részletes tervek.
2. **Költség alapú szerződés**:
   - **Előnyök**: Rugalmas, bővíthető.
   - **Hátrányok**: Költségnövekedés, lassú fejlesztés.
   - **Kockázatcsökkentés**: Agilis módszerek, előzetes költségbecslés.

Kapcsolat a módszertannal

- **Fix költségű**: Vízesés modell.
- **Költség alapú**: Agilis vagy evolúciós megközelítések.

Szoftverfejlesztési módszerek

Szempont	Vízesés modell	RUP	Evolúciós	Agilis
Filozófiai megközelítés	Szekvenciális, merev	Strukturált, alkalmazkodó	Fokozatos fejlődés	Rugalmas, iteratív
Tervezés és követelmények	Átfogó, fix	Strukturált, alkalmazkodó	Kezdeti, változhat	„Just-in-time", rugalmas
Iterációk és kiadások	Egyszeri, nagy kiadás	Iteratív és fokozatos	Hosszabb ciklusok	Rövid iterációk, gyakori kiadások
Ügyfél bevonása	Korlátozott a követelményfázisra	Ügyfél együttműködés	Értékelt, de kevésbé intenzív	Folyamatos, szerves a fejlesztés során
Változáskezelés	Ellenáll a változásoknak a követelményfázis után	Alkalmazkodó a változásokhoz	Alkalmazkodik a változásokhoz	Bármely szakaszban elfogadja a változást
Dokumentáció	Átfogó minden fázisban	Átfogó, alkalmazkodó	Szükség szerint kiterjedt	Minimális, a működő szoftverre összpontosít
Kockázatkezelés	Hagyományos kockázatelemzés minden fázisban	Kockázatkezelést tartalmaz	Az evolúciós kockázatok kezelése	Gyakori alkalmazkodás, folyamatos kockázatkezelés

---

Vízesés modell (Waterfall)

1. **Jellemzők**:
    - Szisztematikus, lineáris modell.
    - Lépések: követelmények → tervezés → megvalósítás → tesztelés → telepítés → karbantartás.
    - Erős dokumentáció, kevés változtatás lehetősége.
2. **Előnyök**:
    - Könnyen tervezhető.
    - Határidők előre becsülhetők.
3. **Hátrányok**:
    - Lassú, nehezen alkalmazkodik változásokhoz.
    - Hosszú fejlesztési fázis, kevés felhasználói visszacsatolás.

V-Modell

- A fejlesztési és tesztelési lépések párhuzamosan haladnak.
- Az egyes fázisokat tesztelési lépések egészítik ki.

RUP módszertan (Rational Unified Process)

1. **Jellemzők**:
    - Iteratív és adaptív, 4 fő fázis:
        - **Inception**: Projekt céljainak meghatározása.
        - **Elaboration**: Architektúra és erőforrások kidolgozása.
        - **Construction**: Implementáció, tesztelés.
        - **Transition**: Átadás, finomítás.
    - Szerepkörök, feladatok és eredmények tiszta definiálása.
2. **Elvek**:
    - Iteratív fejlesztés.
    - Komponensek és vizuális modellezés használata.
    - Minőségellenőrzés és változáskezelés.

Evolúciós modellek (Spirál, Agile)

- **Spirál modell**:
    - Iteratív, kockázatelemzésre fókuszáló fejlesztés.
    - Négy fő fázis: tervezés, kockázatelemzés, implementáció, értékelés.
- **Agilis modellek**:
    - **Jellemzők**: Iteratív, folyamatos prioritás- és irányfelülvizsgálat.
    - **Értékek**:
        - Egyének és interakciók fontosabbak, mint eszközök.
        - Működő szoftver a dokumentáció helyett.
        - Rugalmasan reagál a változásokra.

Az agilis fejlesztés 12 alapelve egyszerűen, pontokban:

1. **Megrendelő elégedettsége**: Értékes szoftvert szállítunk folyamatosan.
2. **Üdvözöljük a változásokat**: A késői változtatási igényeket is kezeljük.
3. **Gyakori kiadás**: Működő szoftvert adunk ki rendszeresen, rövid időszakok alatt.
4. **Együttműködés**: Üzleti szereplők és fejlesztők napi szintű kommunikációja.
5. **Motivált csapat**: A csapatnak biztosítjuk a szükséges környezetet és támogatást.
6. **Személyes kommunikáció**: A leghatékonyabb információáramlás a személyes beszélgetés.
7. **Működő szoftver a mérce**: A haladás mértékét a működő szoftver mutatja.
8. **Fenntartható tempó**: Hosszú távon állandó sebességet tartunk fenn.
9. **Technikai kiválóság**: Jó dizájn és technikai megoldások növelik az agilitást.
10. **Egyszerűség**: Csak a szükséges munkát végezzük el.
11. **Önszerveződő csapat**: A legjobb eredményt az önállóan dolgozó csapatok érik el.
12. **Folyamatos fejlődés**: A csapat rendszeresen megvizsgálja, hogyan javíthatja a munkáját.

SCRUM

1. **Jellemzők**:
    - Iteratív és inkrementális fejlesztés.
    - Fő szerepkörök:
        - **Product Owner (PO)**: Irány kijelölése, követelmények kezelése.
        - **Scrum Master (SM)**: Scrum elvek betartatása, csapat segítése.
        - **Development Team (DT)**: Fejlesztők önszerveződő csoportja.
2. **Események**:
    - **Sprint Planning**: Sprint célok meghatározása.
    - **Daily Scrum**: Napi 15 perces státuszmeeting.
    - **Sprint Review**: Elkészült munkák bemutatása.
    - **Sprint Retrospective**: Munkafolyamatok javítása.
3. **Backlog-kezelés**:
    - **Product Backlog**: Prioritás szerinti feladatok listája.
    - **Sprint Backlog**: Az aktuális sprintre kiválasztott feladatok.